03 – Oprava: vlastní zpráva označila chat jako nepřečtený

1. Co bylo uděláno

Chyba: když jsem napsal zprávu, objevila se mi u té konverzace v seznamu fialová tečka „nepřečteno“, i když šlo o mou vlastní zprávu. Opravil jsem to v databázi: odeslání zprávy teď zároveň posune můj čas „přečteno“.

2. Nové a změněné soubory

Soubor	Co se změnilo
`supabase/migrations/20261016001_chat_sender_read.sql`	Nová migrace, přepisuje funkci triggeru `tg_messages_after_insert`
`docs/database.md`	Popis sloupce `last_read_at`

Frontend se neměnil.

3. Proč chyba vznikla

Nepřečtenost se počítá ve `fetchConversations` (`chat-queries.ts`):

```
unread = last_message_at > můj last_read_at
```

- `last_message_at` posouvá trigger po každém INSERT do `messages` , **bez ohledu na to, kdo zprávu poslal**.
- `last_read_at` posouvala jen RPC `mark_conversation_read` , volaná při otevření konverzace a při **cizí** příchozí zprávě (`useMessages` : `if (msg.sender_id !== myId) markRead(...)`).

Takže: otevřu chat v 14:00 (`last_read_at = 14:00`), v 14:05 napíšu zprávu (`last_message_at = 14:05`). 14:05 > 14:00, takže se konverzace tváří jako nepřečtená.

4. Jak oprava funguje krok po kroku

1. Klient vloží řádek do `messages` .

2. Spustí se AFTER INSERT trigger `tg_messages_after_insert` (SECURITY DEFINER, protože klient nemá právo UPDATE na tyto tabulky).
3. Trigger nastaví `conversations.last_message_at = NEW.created_at` (jako dřív).
4. **Nově** nastaví odesílateli `conversation_participants.last_read_at = NEW.created_at`.
5. Obě změny proběhnou v jedné transakci s insertem. Realtime event dorazí až po commitu, takže když se seznam přenačte, je `last_message_at == last_read_at` a podmínka `<` neplatí, tečka se tedy neobjeví.

`GREATEST(...)` hlídá, aby se `last_read_at` nikdy neposunul zpátky. Systémové zprávy mají `sender_id = NULL`, takže neodpovídají žádnému účastníkovi a nic nemění.

5. Proč v databázi a ne ve frontendu

Šlo by to i v klientovi (např. zjistit `sender_id` poslední zprávy), ale:

- potřeboval bych další data v dotazu na seznam,
- pravidlo „kdo píše, ten četl" by platilo jen v tomhle klientovi, ne třeba v budoucí mobilní aplikaci.

V triggeru platí vždy a pro všechny klienty.

6. Nové pojmy

- **AFTER INSERT trigger**: funkce, kterou Postgres spustí automaticky po vložení řádku. Běží ve stejné transakci.
- `GREATEST(a, b)`: vrátí větší hodnotu. `COALESCE(x, y)` vrátí první hodnotu, která není NULL.
- `CREATE OR REPLACE FUNCTION`: přepíše tělo existující funkce. Trigger samotný se znovu vytvářet nemusí, protože odkazuje na funkci jménem.

7. Co si mám sám vyzkoušet a ověřit

1. Spustit migraci v Supabase (SQL editor nebo `supabase db push`).
2. Přihlásit se jako A, napsat zprávu B a ověřit, že se u A tečka **neobjeví**.
3. V anonymním okně jako B ověřit, že se u B tečka **objeví**, a po otevření konverzace zmizí.
4. V tabulce `conversation_participants` se podívat, že `last_read_at` odesílatele je stejný jako `created_at` jeho zprávy.

8. Kontrolní otázky

1. Proč porovnání `last_read_at < last_message_at` po opravě nevrátí `true`, když jsou obě hodnoty stejné?
2. Proč musí být funkce triggeru `SECURITY DEFINER`?
3. Co by se stalo bez `GREATEST`, kdyby zpráva měla starší `created_at` než aktuální `last_read_at`?

4. Proč se tečka neobjeví ani na okamžik, i když seznam reaguje na realtime INSERT do `messages`?